

ICT 1306

OBJECT ORIENTED PROGRAMMING

2. INTRODUCTION TO C++

TC Irugalbandara
KHA Hettige

1. Basic Program Construction

```
/* Name: comments.cpp
```

```
Purpose: demonstrates comments
```

```
Author : ABC
```

```
Version : 1 */
```

```
#include <iostream> // preprocessor directive. tells the compiler to insert another  
file into your source
```

```
using namespace std; // using directive.
```

```
int main() {
```

```
    cout << "Every age has a language of its own\n";
```

```
    // std::cout << "Every age has a language of its own.";
```

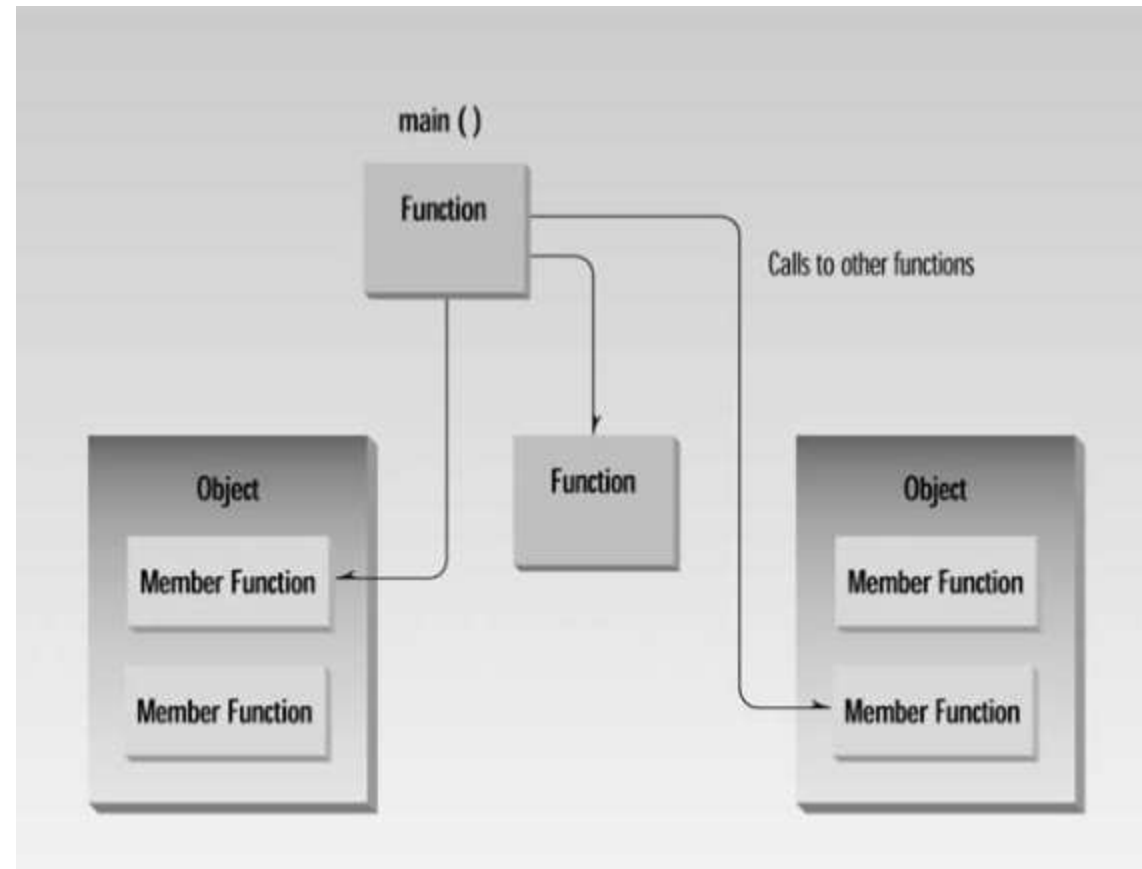
```
    return 0;
```

```
}
```

- cout is a predefined Object .

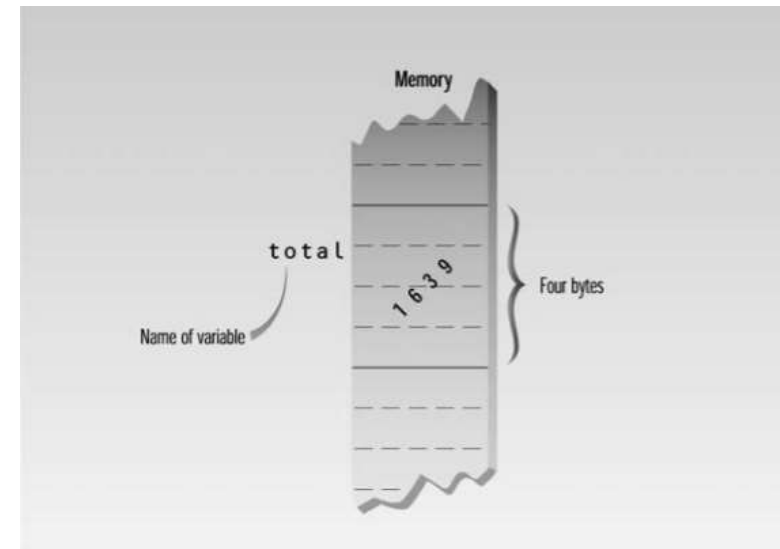
Functions

- The parentheses following the word function name are the distinguishing feature of a function.
- Parentheses aren't always empty. They're used to hold function arguments: values passed from the calling program to the function.
- Return type.
- The body of a function is surrounded by braces (sometimes called curly brackets).
- Control always goes to `main()` at startup.



Data Types and Variables

- A variable has a symbolic name and can be given a variety of values.
- Variables are located in particular places in the computer's memory.
- When a variable is given a value, that value is actually placed in the memory space assigned to the variable.
- You must declare a variable before using it



- Name given to a variable is called its identifier.
- You can't use a C++ keyword as a variable name. A keyword is a predefined word with a special meaning.
 - int, class, if, and while .
- Int the memory space may differ based on the type of the system.
 - 32 bit – 4 bytes / 16- bit – 2 bytes
- long → 4 bytes
- short → 2 bytes
- Char,float,double,long double, bool

- Take two integer inputs from the user , add them together and divide the result by 3. Display the answer .

Type Conversion

// shows mixed expressions

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int count = 7;
```

```
    float avgWeight = 155.5F;
```

```
    double totalWeight = count * avgWeight;
```

```
    cout << "totalWeight=" << totalWeight << endl;
```

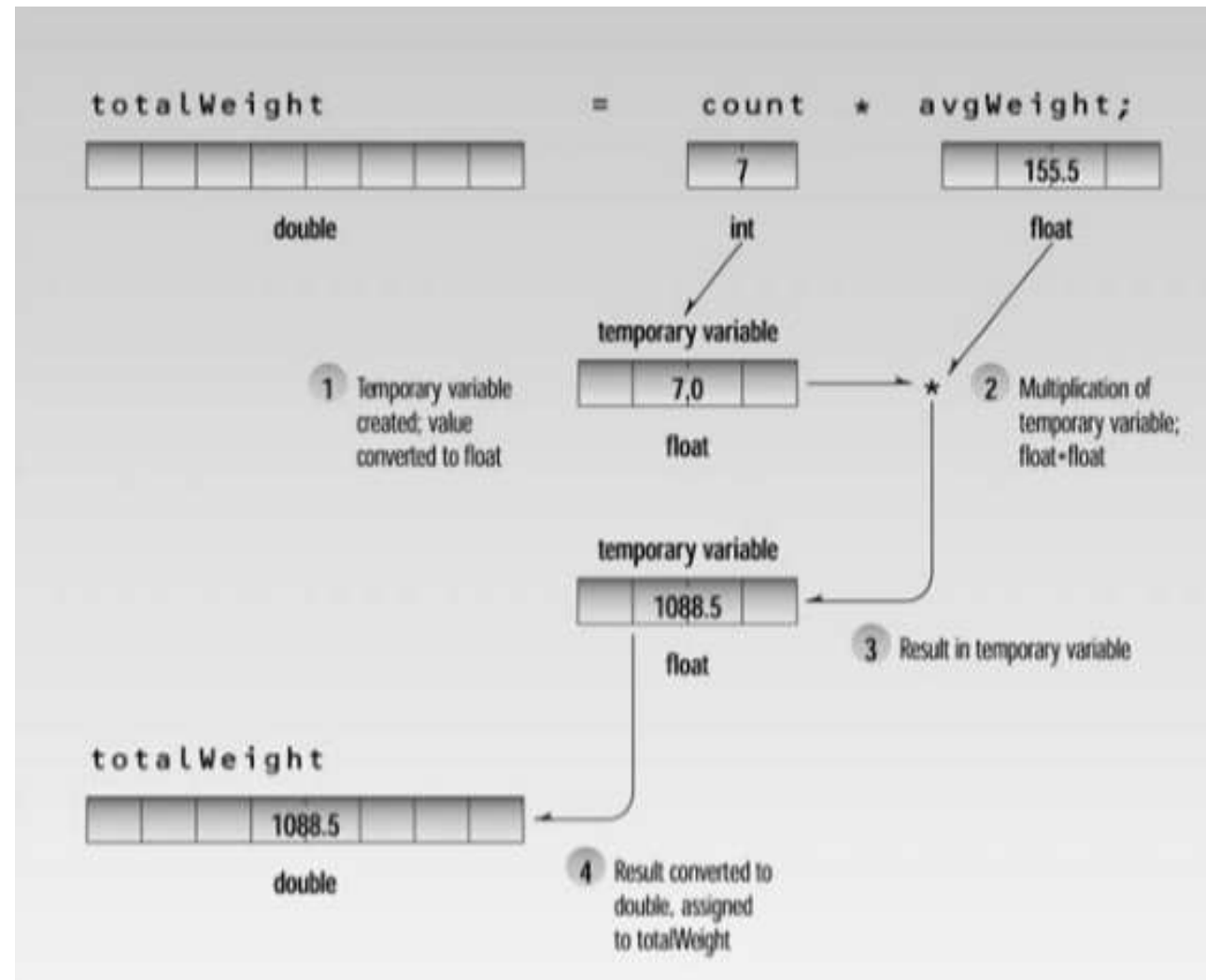
```
    return 0;
```

```
}
```

C++ same as C gives more freedom to mix the types.

<i>Data Type</i>	<i>Order</i>
long double	Highest
double	
float	
long	
int	
short	Lowest
char	

- When two operands of different types are encountered in the same expression, the lower-type variable is converted to the type of the higher-type variable.
- We will defining our own data types. We may want to use these new data types in mixed expressions. ???



Casts

- Data conversions specified by the programmer.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int intVar = 1500000000;           //1,500,000,000
```

```
    intVar = (intVar * 10) / 10;        //result too large
```

```
    cout << "intVar = " << intVar << endl; //wrong answer
```

```
    intVar = 1500000000;               //cast to double
```

```
    intVar = (static_cast<double>(intVar) * 10) / 10;
```

```
    cout << "intVar = " << intVar << endl; //right answer return 0;
```

```
}
```

Arithmetic Operators

Arithmetic Assignment Operators

`total = total + item;`

- Combines an arithmetic operator and an assignment operator and eliminates the repeated operand.

`total += item;`

Ex: Define a variable of type int and initialize it to 25. Use arithmetic assignment operators and

- add 5 to it,
- subtract 6 from it ,
- divide by 4 and
- multiply by 3 and store the result in the same variable each time.

Increment Operators

- You often need to add 1 to the value of an existing variable. You can do this the “normal” way:

```
count = count + 1; // adds 1 to “count”
```

- Or you can use an arithmetic assignment operator:

```
count += 1; // adds 1 to “count”
```

- But there’s an even more condensed approach:

```
++count; // adds 1 to “count”
```

- The ++ operator increments (adds 1 to) its argument.

Prefix and Postfix

- The increment operator can be used in two ways:
- as a prefix, meaning that the operator precedes the variable;
- and as a postfix, meaning that the operator follows the variable.

```
totalWeight = avgWeight * ++count;
```

```
totalWeight = avgWeight * count++;
```

The decrement operator, --, behaves very much like the increment operator, except that it subtracts 1 from its operand.

- What is the output ???

```
#include <iostream>
using namespace std;
int main() {
    int count = 10;
    cout << "count=" << count << endl; //displays 10
    cout << "count=" << ++count << endl; //displays 11 (prefix)
    cout << "count=" << count << endl; //displays 11
    cout << "count=" << count++ << endl; //displays 11 (postfix)
    cout << "count=" << count << endl; //displays 12 return 0;
}
```

2. Loops and Decisions

- Control statements. There are two major categories: loops and decisions.
- How many times a loop is executed, or whether a decision results in the execution of a section of code, depends on whether certain expressions are true or false.
- These expressions typically involve a kind of operator called a relational operator, which compares two values.

<i>Operator</i>	<i>Meaning</i>
>	Greater than (greater than)
<	Less than
==	Equal to
!=	Not equal to
>=	Greater than or equal to
<=	Less than or equal to

Loops : The “for” Loop

- The for loop executes a section of code a fixed number of times.
- It's usually (although not always) used when you know, before entering the loop, how many times you want to execute the code.

```
int main() {  
    int j;           //define a loop variable  
    for(j=0; j<15; j++) //loop from 0 to 14,  
        cout << j * j << " "; //displaying the square of j  
    cout << endl;  
    return 0;  
}
```

```
for(j=0; j<15; j++)
```

- These three expressions are the initialization expression, the test expression, and the increment expression.
- The initialization expression is executed only once, when the loop first starts. It gives the loop variable an initial value.
- The test expression usually involves a relational operator. It is evaluated each time through the loop, just before the body of the loop is executed. It determines whether the loop will be executed again. If the test expression is true, the loop is executed one more time. If it's false, the loop ends, and control passes to the statements following the loop.
- The increment expression changes the value of the loop variable, often by incrementing it.

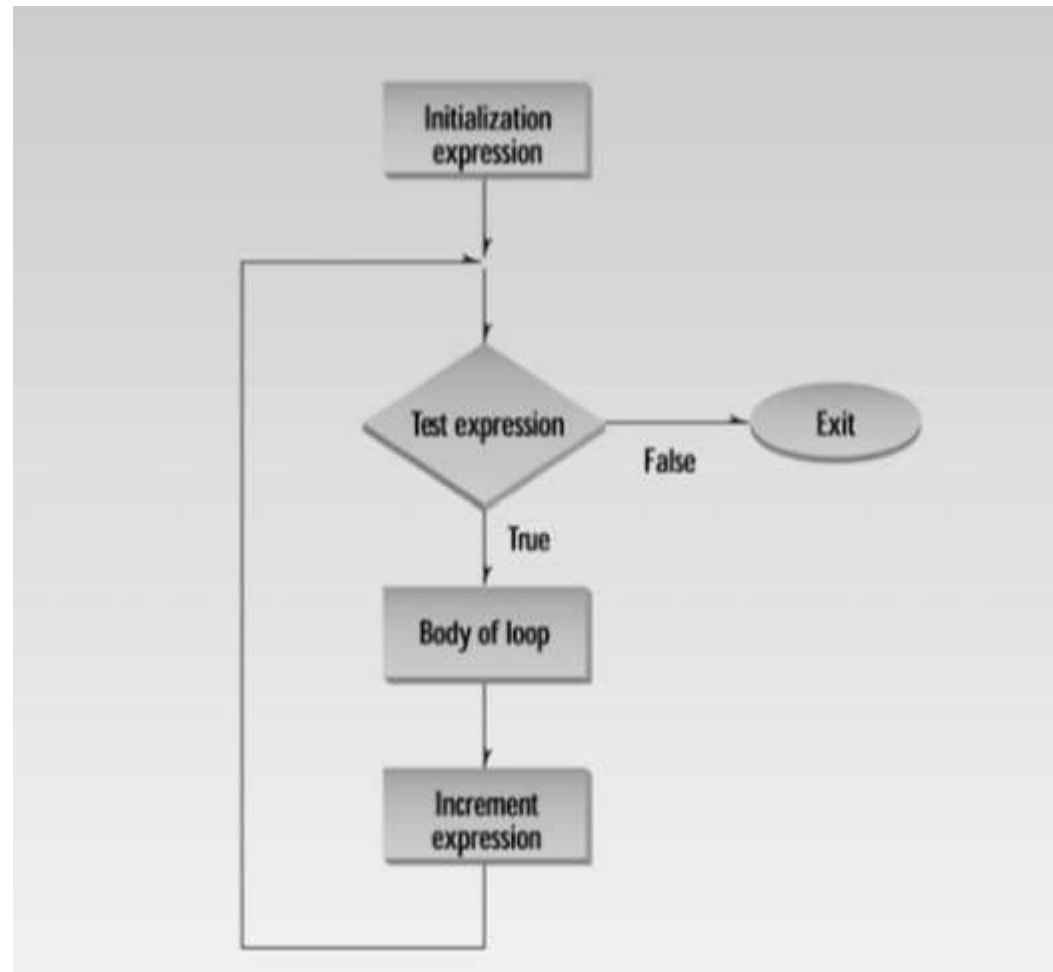
- ex: Ask the user to enter a series of numbers. When the number entered is 0, the loop terminates. Write a C++ programme.

```
// endon0.cpp
// demonstrates WHILE loop
#include <iostream>
using namespace std;

int main()
{
    int n = 99;          // make sure n isn't initialized to 0

    while( n != 0 )      // loop until n is 0
        cin >> n;        // read a number into n
    cout << endl;
    return 0;
}
```

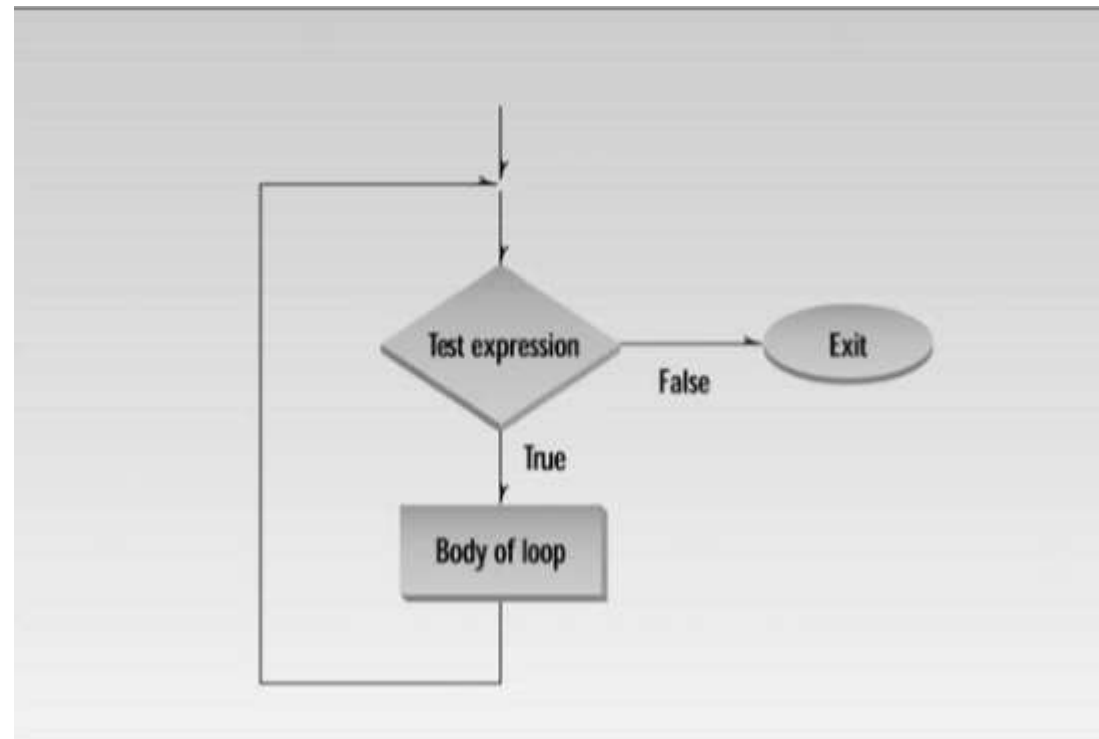
- Draw a flowchart to the for loop execution.



- Write a C++ programme to print numbers from 1 to a user given value (x).
- Write a C++ programme to calculate the factorial of any number.

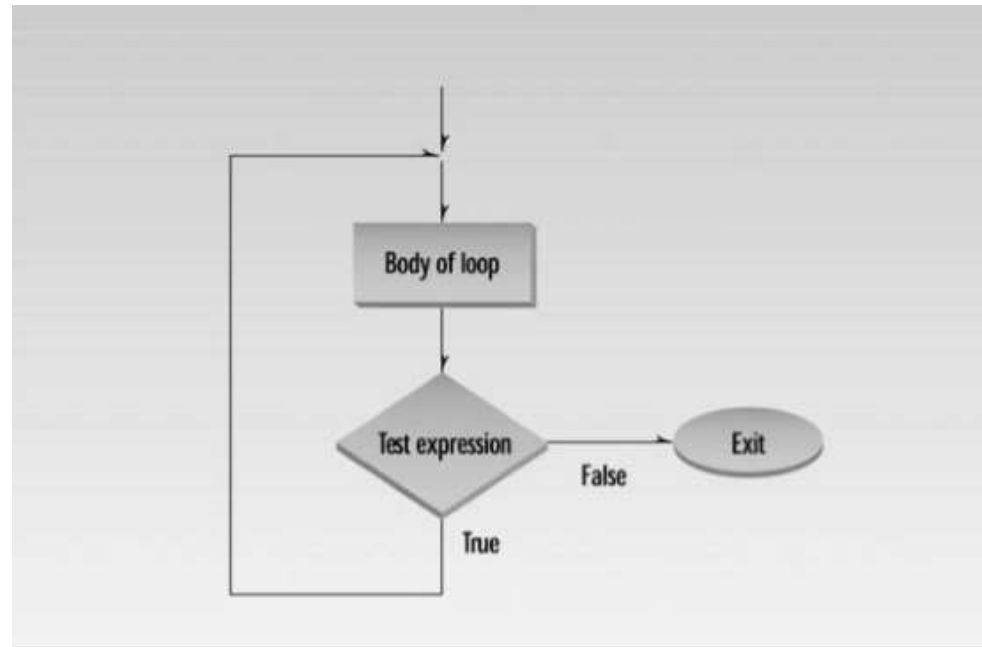
Loops : The “while” Loop

- If you don't know how many times you want to do something before you start the loop .
- The while loop looks like a simplified version of the for loop. It contains a test expression but no initialization or increment expressions



Loops : The “do while” Loop

- you want to guarantee that the loop body is executed at least once, no matter what the initial state of the test expression.
- This while statement looks much like the one in a while loop, except for its position at the end of the loop and the fact that it ends with a semicolon



do  — Note: no semicolon here
 statement;  Single-statement loop body
while (ch != 'n');
 Test expression  Note: semicolon

do  — Note: no semicolon here
 {
 statement;
 statement;
 statement;
 }
while (numb < 96);
 Test expression  Note: semicolon  Multiple-statement loop body

- Write a C++ programme to take 2 user inputs , add them and divide the result by 2 and print the result and the remainder , continue this until user need to stop it.

Decision

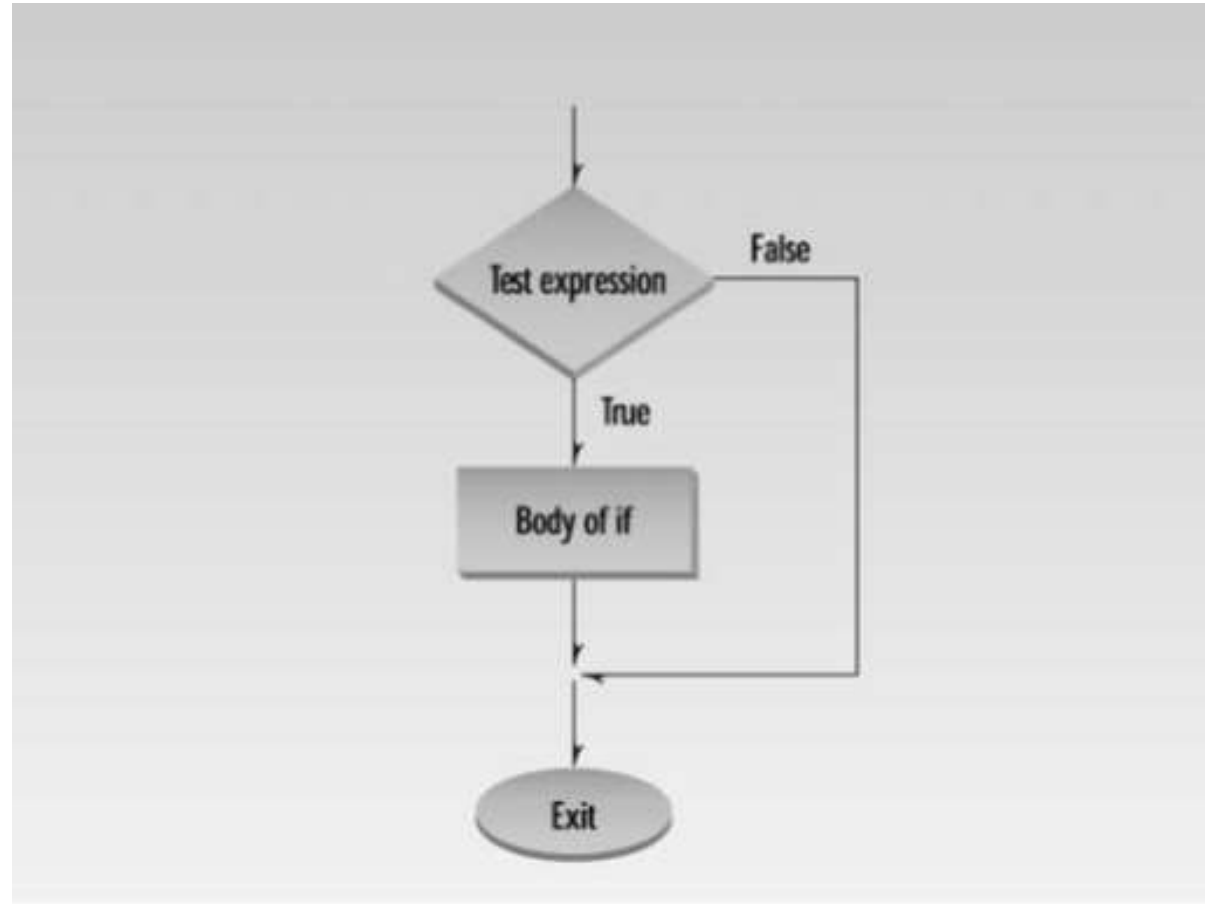
- A decision causes a onetime jump to a different part of the program, depending on the value of an expression
 - If.. else
 - Switch
 - Conditional operator

if

Test expression
if (x>100)
statement; Single-statement if body

Test expression
if (speed<=55)
{
statement;
statement;
statement;
}
Note: no semicolon here Multiple-statement if body

Operation of the “if” statement



- Ex: Take a number if it is greater than 25 do something if it is not do something else.

If.... else

Test expression

```
if (x>100)
    statement;
else
    statement;
```

Single-statement if body

Single-statement else body

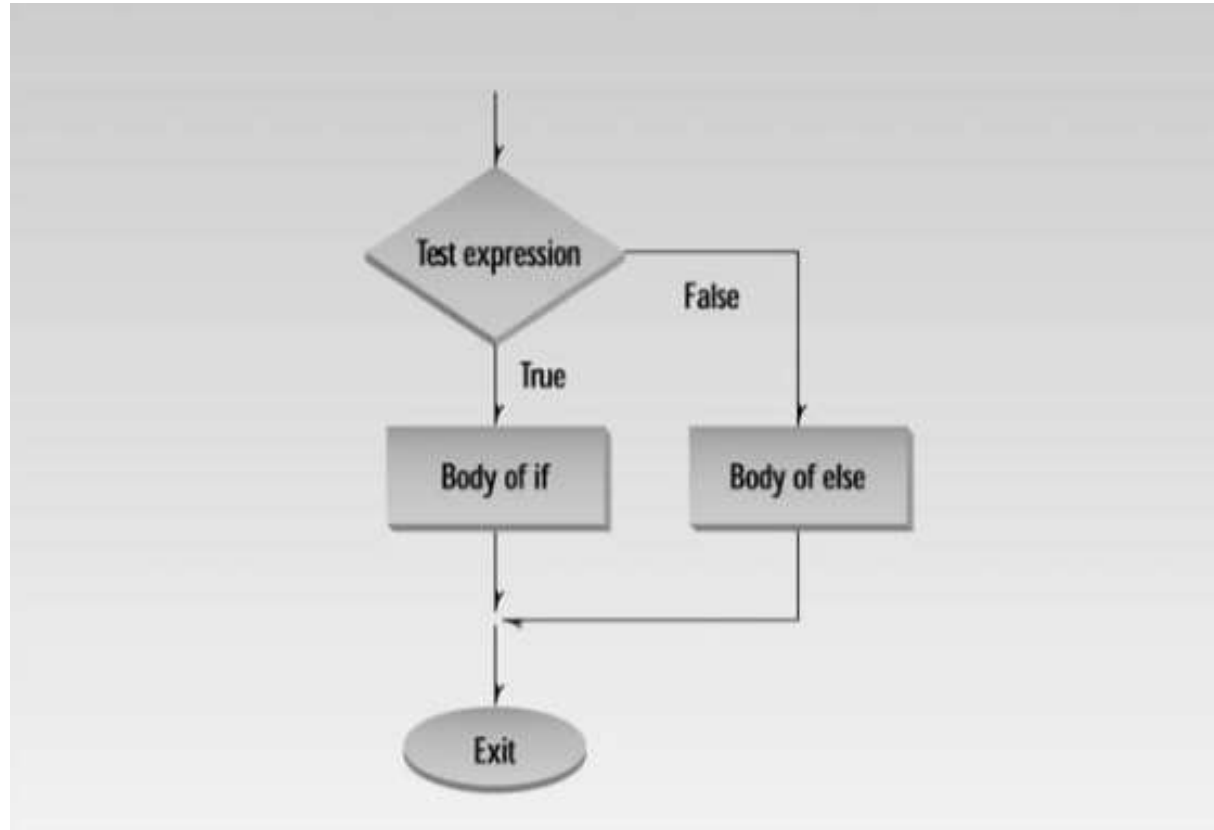
Test expression

```
if (zebra!=0)
{
    statement;
    statement;
}
else
{
    statement;
    statement;
}
```

Multiple-statement if body

Multiple-statement else body

Operation of “if.. else” statement



- Ex: Take a number if it is below 10 print stars(*) if it is not print @.

Nested if...else Statements

- Ex: Ask the user to input his grade mark(X), print
 - If $X > 85 \rightarrow A$
 - $X > 75 \rightarrow B$
 - $X > 65 \rightarrow C$
 - Else D

The switch Statement

- If you have a large decision tree, and all the decisions depend on the value of the same variable, you will probably want to consider a switch statement .

```
switch (n) {  
    case 1:  
        statement;  
        statement;  
        break;  
    case 2:  
        statement;  
        statement;  
        break;  
    case 3:  
        statement;  
        statement;  
        break;  
    default:  
        statement;  
        statement;  
}  
Note: no semicolon here
```

Integer or character variable

Integer or character constant

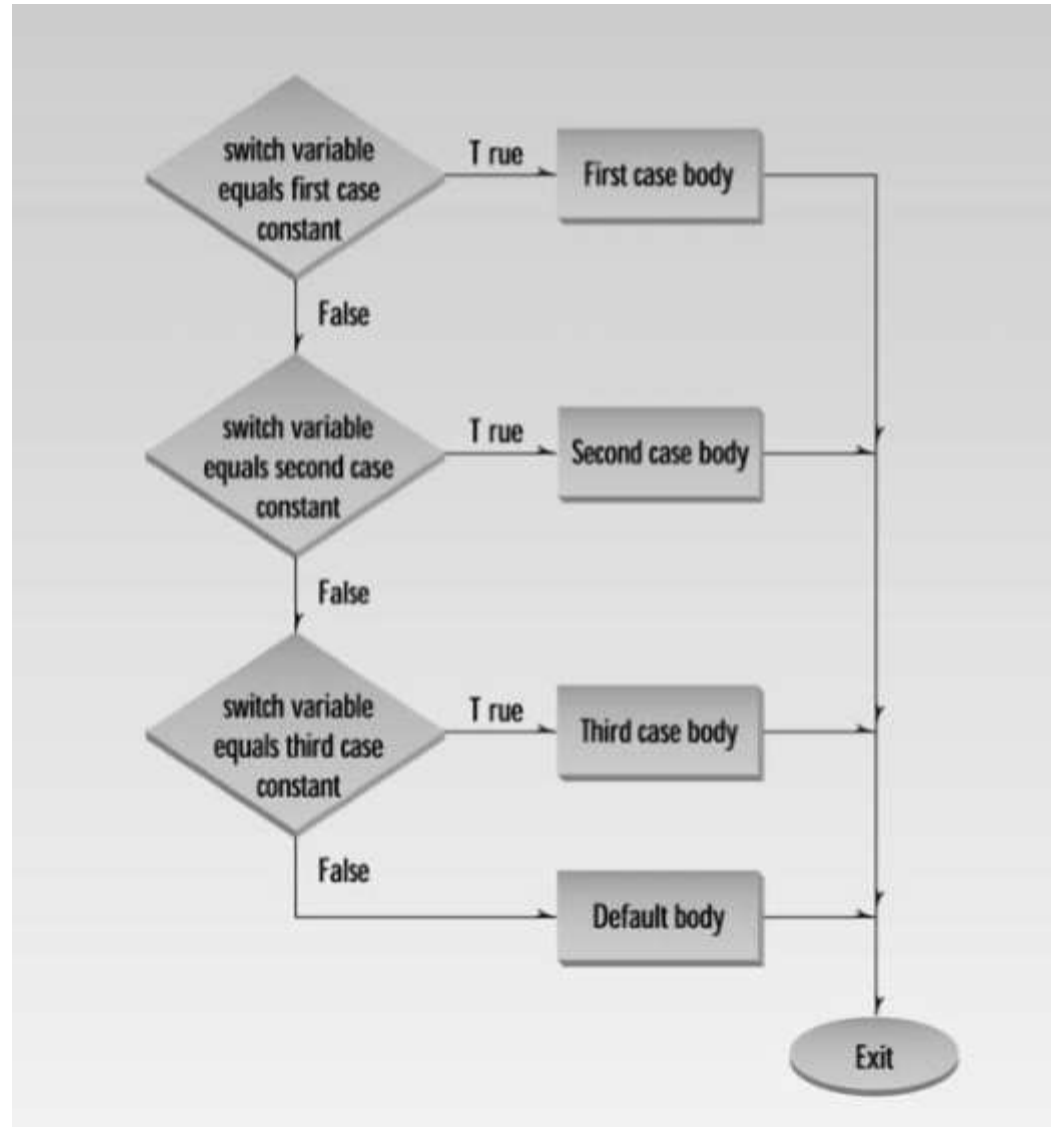
First case body

Second case body

Third case body

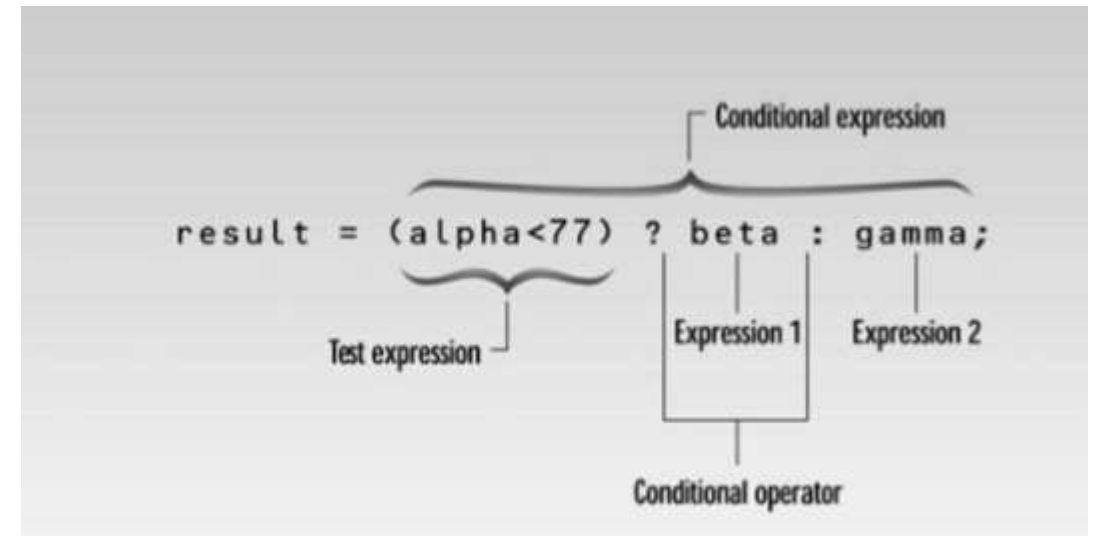
Default body

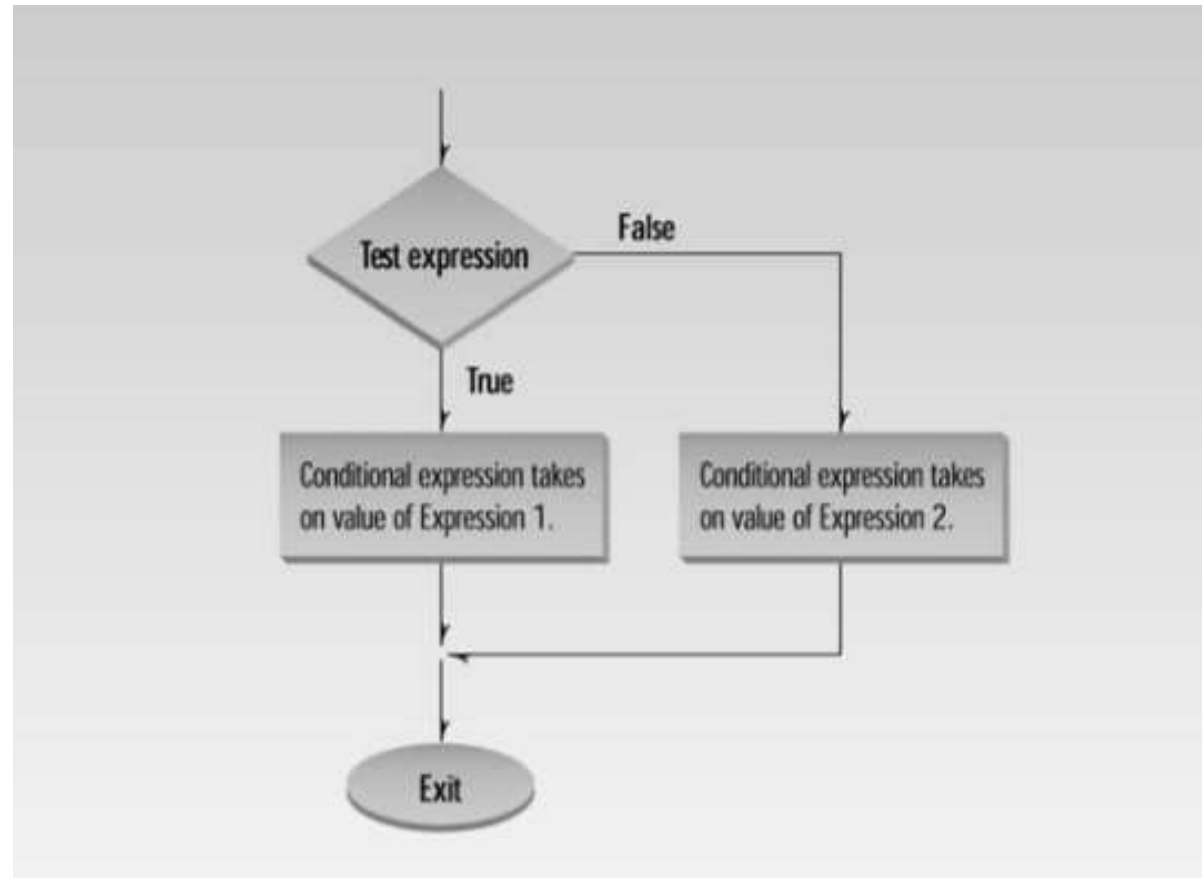
causes exit from switch



Conditional Operator

- A variable is given one value if something is true and another value if it's false.
if(alpha < beta)
 min = alpha;
else min = beta;
- min = (alpha < beta) ? alpha : beta;





Logical Operators

- &&
 - ||
 - !
-
- Write a programme ;
 - Take 2 user inputs(x,y).
 - If if the both 1st num and 2nd num are prime numbers ; print "Both x and y are primes"
 - If one of them are a prime ; print"Only Num x is a Prime"
 - If none of them is a prime; print"Both are not primes".

Thank You !